

Swami Sahajanand College of Computer Science

B.C.A. SEM-VI

Subject: RDBMS using Oracle-II

UNIT 2

Advance PL/SQL Programming

- ◆ Exception Handling
- ◆ Stored Procedure and Function
- ◆ Trigger
- ◆ Data Concurrency and locking

Q-1 Explain Exception Handling in Oracle.**❖ What is Exception?**

- ❑ Exception is a one type of error.
- ❑ Exception can be user defined or System Exceptions.
- ❑ Exceptions are automatically generated by PL/SQL.

❖ System Exceptions:

- **Cursor_Already_Open(ORA –06511)**
 - When user tries to open a cursor which is already open this error is generated.
- **Invalid_Cursor(ORA –01001):**
 - When user performs any invalid cursor operation this error is generated.
- **Dup_Val_On_Index (ORA –00001)**
 - When user tries to enter a duplicate value in a unique column this error is raised.
- **Invalid_Number (ORA –01722)**
 - When user tries to enter other than numeric value in numeric field this error is generated.
- **Login_Denied(ORA –01017)**
 - When user tries to login with invalid User name & Password this exception will be generated.
- **No_Data_Found(ORA –01403)**
 - When single row select statement found no data then this error will be generated.
- **Not_Logged_On(ORA –01012)**
 - This exception is generated when user is not connected to oracle.
- **Program_Error(ORA –06501)**
 - When PL/SQL block has an internal error this error is generated.
- **Storage_Error(ORA –06500)**
 - When PL/SQL block has insufficient memory to execute this error is generated.
- **Too_Many_Rows(ORA –01422)**
 - This error will be generated when single row select statement generates more than one row.
- **Value_Error(ORA –06502)**
 - This exception is raised whenever the user encounters an arithmetic, conversion or constraints error.
- **Zero_Divide(ORA –01476)**
 - This error is raised whenever PL/SQL block tries to divide any value by zero.

:: Example of System Generated Exception:

```
declare
    salary number;
begin
    select sal into salary from emp where empno=100;
exception
    when no_data_found then
        dbms_output.put_line("No records are found for empno 100");
end;
```

❖ User Defined Exceptions:

- User defined exceptions must be declared in the DECLARE section with the reserve word EXCEPTION.
<EXCEPTION_NAME> EXCEPTION;
- This exception can be brought into action by the following command in executable section.
RAISE <EXCEPTION_NAME>;
- When the exception is raised, processing control is passed to the EXCEPTION section of the PL/SQL block.
- Therefore, the code for the exception must be defined in the EXCEPTION part of the PL/SQL block.
WHEN <EXCEPTION_NAME> THEN
<ACTION>;

:: Example of User Defined Exceptions::

```

Declare
comission number (5,2);
      null_comission exception;
begin
      select emp_comission into comission from emp where emp_no = 100;
      if comission is null then
            raise null_commission;
      end
if;
exception
      when null_comission then
            dbms_output.put_line('no comission found for aiven employee');

```

Q-2 Explain Stored Procedure and Function with example.

- ❏ A stored procedure or functions is a logically grouped set of sql and pl/sql statements that perform a specific task.
- ❏ This is similar to a procedure/function in other programming languages.
- ❏ A stored procedure/function has a header and a body.
- ❏ The header consists of the name of the stored procedure/function and the parameters or variables passed to the stored procedure/function.
- ❏ The body includes declaration section, execution section and exception section similar to a general PL/SQL Block.
- ❏ There are important difference between anonymous block and procedure is that:
 - ✓ Anonymous blocks stored as file & must be compiled and then run each time the script is loaded while stored procedure/function is stored as object in database in binary form so it is compiled only once when it is created.
 - ✓ Anonymous block cannot accept parameter as well as cannot return any value to and from external application while stored procedure/function can accept parameter as well as return value to and from external application.
- ❖ **Advantage of using procedure and function:**
 - **Security:**
 - It provides data security.
 - Such as by giving permission to a procedure or function that can query a table and granting the procedure to users, not need permission to manipulate the table itself.
 - **Performance:**
 - Amount of information sent over a network is less.
 - No compilation is required to execute the code.
 - Once the procedure is present in the shared pool of the SGA retrieval from disk is not required every time different users call the procedures or function. Such as reduction in disk I/O.
 - **Memory allocation:**
 - It provides shared memory facility.
 - Only one copy of procedure needs to be loaded for execution by multiple users.
 - Once a procedure is opened in the Oracle engine's memory, other users who have appropriate permission may access it when required.
 - **Productivity:**
 - By writing procedures repetitive coding can be avoided so increasing the productivity.

- **Accuracy:**

- A procedure needs to be tested only once to guarantee that it returns an accurate result.

:: Syntax to create stored procedure ::

```
CREATE OR REPLACE PROCEDURE < procedure name >
(<argument> {IN/ OUT/ IN OUT} <DATA TYPE>...) IS.AS}
<variable> declarations:
<constant> declarations:
BEGIN
<PL/SQL subprogram body>:
EXCEPTION
<exception PL/SQL block>:
END:
```

:: Example of stored procedure::

create or replace procedure delete_emp(eno in number) is

```
BEGIN
delete from emp where empno= eno;
END;
```

:: How to execute procedure? ::

```
start procedure1.sql;
execute delete_emp(10);
```

:: To Drop Procedure ::

Syntax: drop procedure <procedure_name>;
Example: drop procedure delete_emp;

❖ Keywords and parameters used in Procedure / Function:

Keywords	Description
Replace	Recreates the procedure if already exists. If a procedure is redefined the Oracle engine recompiles it.
Schema	Give the schema name which contains the procedure. The oracle engine takes the default schema to be the current schema, if it is omitted.
Procedure	It Is the name of the procedure to be created.
Argument	It Is the name of an argument to the procedure. Parentheses can be omitted if no arguments are present.
IN	Indicates that the parameter will accept value from user.
OUT	Indicates that the parameter will return a value to the user.
IN OUT	Indicates that the parameter will either accept a value from the user or return a value to the user.
Data type	It Is the data type of an argument. It supports any data type supported by PL/SQL.

:: Example of stored procedure ::

```
create or replace procedure find_name(v_empno in number) as
    v_ename emp.ename%type;
    v_job emp.job%type;
    v_sal emp.sal%type;
begin
    select ename,job,sal into v_ename,v_job,v_sal from emp where empno=v_empno;
    dbms_output.put_line(' Name ' || v_ename || ' designation ' || v_job || ' salary ' || v_sal);
exception
    when NO_DATA_FOUND then
        dbms_output.put_line(' Record not found, please try again');
end;
```

Sample stored procedure that has in & out parameter

```
create or replace procedure addvalue(num1 in number,num2 in number, answer out number) is
begin
    answer:=num1+num2;
    dbms_output.put_line('answer of addition is :- ' || answer);
end;
```

- ❏ If stored procedure has out parameter, then it should be called from either other stored procedure/function or anonyms block.
- ❏ Above created “addvalue” stored procedure can be called from anonyms block in following way

```
Set serveroutput on;
declare
    ans number(5,2);
begin
    addvalue(10,20,ans);
    dbms_output.put_line('answer of addition is ' || ans);
end;
```

:: Syntax to create Function ::

```
CREATE OR REPLACE FUNCTION < Function name >
    (<argument> {IN/ OUT/ IN OUT} <DATA TYPE>...) return <data type>{IS.AS}
<variable> declarations:
<constant> declarations:
BEGIN
    <PL/SQL subprogram body>:
EXCEPTION
    <exception PL/SQL block>:
```

:: Example of function that returns value ::

```

create or replace function factorial(num in integer) return integer as
    ans integer(5) :=1;
begin
    for i in 1..num loop
        ans:=ans*i;
    end loop;
    return ans;
end;
```

:: Function called from another anonyms block ::

```

set serveroutput on;
declare
    answer integer(5) :=1;
begin
    answer:= factorial(5); dbms_output.put_line('answer is :-' || answer);
end;
```

❖ Difference between procedure & function:

Procedure	Function
Not require to return value.	Must require to return value.
Using multiple OUT parameters we can return multiple values.	It returns only single value at a time.
Procedure keyword is used.	Function keyword is used.

Q-3 What is Trigger? Explain components and types of trigger.

- ❑ A database trigger is a named PL/SQL block stored in a database and executed implicitly (automatically) when a specific event occurs.
- ❑ An act of executing a trigger is referred to as firing a trigger.
- ❑ A triggering event is a DML (INSERT, UPDATE, or DELETE) statement executed against a database table.
- ❑ A trigger is a compiled procedure stored in the database.
- ❑ The language you use is PL/SQL. You code and compile a trigger in the same manner you code stored procedures.
- ❑ A trigger can fire before or after a triggering event.
- ❑ For example, if you have defined a trigger to fire before an INSERT statement on the STUDENT table, this trigger fires each time before you insert a row in the STUDENT table.
- ❑ The general syntax for creating a trigger is as follows (the reserved words and phrases surrounded by brackets are optional):

❖ Components of Triggers:

- **Triggering event or statement:**
 - It is a SQL statement that causes a trigger to be fired.
 - INSERT, UPDATE, or DELETE statement for a specific table.
- **Trigger Restriction:**
 - It specifies a Boolean expression that must be TRUE for the trigger to fire.
 - It is option available for triggers that are fired for each row.
 - Its function is to conditionally control the execution of trigger.
 - Trigger restriction is specified using a WHEN clause.

- **Trigger Action:**

- Trigger action is the PL/SQL code to be executed when a triggering statement is encountered and any trigger restriction evaluates to TRUE.
- It contains SQL & PL/SQL statements.

- ❖ **Types of Triggers**

- **Rows triggers:**

- It is fired each time a row in the table is affected by the triggering statement.
- For example if update statement updates multiple rows of a table, a row trigger is fired once for each row affected by update statement.
- If triggering statement affects no rows, the trigger is not executed at all.
- It is used when some processing is required whenever a triggering statement affects a single row in a table.

- **Statement triggers:**

- Statement level triggers are triggered only once.
- For example, when an update command updates 15 rows, the commands contained in the trigger are executed only once, and not with every processed row.

- **Before triggers:**

- It executes the trigger action before the triggering statement.
- It is used in the following situation....
 - ✓ User can eliminate unnecessary processing of the triggering statement.
 - ✓ It is used to derive specific column values before completing a triggering INSERT or UPDATE statement.

- **After triggers:**

- After trigger executes the trigger action after the triggering statement is executed.
- It is used in the following situation...
 - ✓ After triggers are used when the triggering statement should complete before executing the trigger action.
 - ✓ If a before trigger is already present, an after trigger can perform different actions on the same triggering statement.

- ❖ **Point to remember:**

- There can be only one trigger of particular type that is for UPDATE, for INSERT or for DELETE.
- Only one table is specified in the triggering event.
- Triggers cannot include commit, rollback and savepoint statement.
- Inside the trigger the correlation name :NEW and :OLD can be made use of to refer to data on the command line and data in the table respectively.

:: Syntax to create a Trigger ::

```
create or replace trigger <trigger_name>
{before/after}
{insert/update/delete} of column on <table_name>
[For each row]
[When condition]
[PL/SQL block];
```

:: Create Table emp10 ::

Create table emp10 (Eno number (7), Name varchar2 (30), Salary number (7,2));

Table creterd...

:: Example of Trigger ::

```
edit emp_delete.sql
set serveroutput on
set verify off
create or replace trigger emp_delete before delete on emp for each row
declare
    eno emp.empno % type;
    name emp.ename % type;
    salary emp.sal % type;
begin
    eno:=:old.empno;
    name:=:old.ename;
    salary:=:old.sal;
    insert into emp10 values(en,name,salary);
end;
```

:: Execution of trigger::

```
start emp_delete.sql; trigger
is created.....
delete from emp where ename='TURNER';
select * from emp10;
```

Q-4 Explain Data Concurrency and Locking in detail.

- ❑ Locks are mechanisms used to ensure data integrity while allowing maximum concurrent access of data.
- ❑ Oracle locking is fully automatic & requires no user intervention.
- ❑ The oracle engine (server machine) locks table data while executing SQL statement. This type of locking is called "implicit locking".
- ❑ Oracle default locking strategy is implicit locking.
- ❑ Since the oracle engine has a fully automatic strategy, it has to decide on two issues:-
 - 1) Types of lock to be applied.
 - 2) Level of lock to be applied.

❖ Level of Locks:

- 1) Row Level:-If the Where clause evaluates to only one row in the table.
- 2) Page Level: - If the Where clause evaluates to a set of data.
- 3) Table Level: - If there is no Where clause (i.e. the query accesses the entire table).

❖ Explicit Locking:

- The technique of lock taken on a table or its resources by a user is called "Explicit Locking".
- Users can lock tables they own or any table on which they have been granted table privilege(select ,insert, update , delete)
- Table or rows can be explicitly locked by using either the select ...for update stmt. Or Lock table stmt.

➤ **The select For Update statement:-**

- This clause is generally used to signal the oracle engine that data currently being used needs to be updated.
- Ex:-Two client machines client A & client B are recording the transaction performed in a bank for a particular account no. simultaneously.
- Client A fires the following select statement:
- Client A>select * from acct_mstr where acct_no='sb9' for update;
- When the above select statement is fired the oracle engine locks the record 'sb9'.
- This lock is released when a commit or rollback is fired by client A.
- Now client B fires a select stmt., which points to record sb9.

Using Lock table statement:-

- Use the LOCK TABLE statement to lock one or more tables, table partitions, or table sub partitions in a specified mode. This lock manually overrides automatic locking and permits or denies access to a table or view by other users for the duration of your operation.
- A **LOCK** is a mechanism that prevents destructive interaction between two simultaneous transactions or sessions trying to access the same database object.

Syntax:-

```
LOCK TABLE<TableName>[,<TableName>]...  
IN{ROW SHARE|ROW EXCLUSIVE|SHARE  
UPDATE|  
    SHARE|SHARE ROW  
    EXCLUSIVE|EXCLUSIVE} [NOWAIT]
```

Example:-

The following statement locks the employees table in exclusive mode but does not wait if another user already has locked the table:

```
LOCK TABLE employees IN EXCLUSIVE MODE NOWAIT;
```

Output:- Table Locked.

❖ **Lock mode Clause:**

Specify one of the following modes:

- **ROW SHARE:**-ROW SHARE permits concurrent access to the locked table but prohibits users from locking the entire table for exclusive access. ROW SHARE is synonymous with SHARE UPDATE, which is included for compatibility with earlier versions of Oracle Database.
- **ROW EXCLUSIVE:**-ROW EXCLUSIVE is the same as ROW SHARE, but it also prohibits locking in SHARE mode. ROW EXCLUSIVE locks are automatically obtained when updating, inserting, or deleting.SHARE UPDATE See ROW SHARE.
- **SHARE:**- SHARE permits concurrent queries but prohibits updates to the locked table.
- **SHARE:**- ROW EXCLUSIVE SHARE ROW EXCLUSIVE is used to look at a whole table and to allow others to look at rows in the table but to prohibit others from locking the table in SHARE mode or from updating rows.
- **EXCLUSIVE:**- EXCLUSIVE permits queries on the locked table but prohibits any other activity on it.

- **NOWAIT** :- Specify NOWAIT if you want the database to return control to you immediately if the specified table, partition, or table sub partition is already locked by another user. In this case, the database returns a message indicating that the table, partition, or sub partition is already locked by another user.
- If you omit this clause, then the database waits until the table is available, locks it, and returns control to you.

❖ **Releasing locks:**

All locks are released under the following circumstances:

- 1) The transaction is committed successfully.
- 2) A rollback is performed
- 3) A rollback to a savepoint will release locks set after the specified savepoint.

Note: - commit: Save Work done.

Savepoint: Identify a point in a transaction to which you can later rollback.

Rollback: Restore database to original since the last COMMIT.

GRANT/REVOKE: Grant or back permission to or from the oracle users.